The FLOPs You Save Are Not the Time You Spend: Sparse Attention in Multi-Agent Communication

Anonymous authors
Paper under double-blind review

Abstract

Attention-based inter-agent communication aggregates over all $N-1$ peers per agent per step, and restricting each agent to its top-k peers by attention score is an appealing way to make that cheaper — a case normally made by counting operations rather than by measuring time. We built one and report why we abandoned it, at three levels.

First, an analytic ceiling. In *score-then-select* top-k attention, peer selection requires the complete score vector, so all candidate scores must first be computed. Only the value-aggregation product — one of the two equal terms in the attention multiply–accumulate count — is sparsifiable, and the reduction is bounded above by $2\times$ for every k and N, not the $1-k/(N-1)$ the aggregation term alone suggests. Selection that fixes the retained set before the scores exist escapes the bound: with selector cost $C_{\text{sel}} = o(Nkd)$ the count ratio is asymptotic to N/k, exactly N/k only when the selector is free.

Second, what executes. Applying top-k as a *mask*, as our first implementation did, leaves the dense aggregation MAC count unchanged and adds selection and masking overhead, while a FLOP model based on $1-k/(N-1)$ reports a 75% saving at our headline N=512, k=127 cell. We implement the genuinely sparse gather instead.

Third, what it costs. Across a head-width $\times$ batch $\times$ N $\times$ k sweep on one GPU architecture the gathered path is slower than unfused dense in all 46 cells; the ordering also holds on a CPU whose machine balance differs by an order of magnitude. A component decomposition and a roofline analysis explain why: at N=512 the sparse path moves $3.8\times$ the traffic for $0.6\times$ the arithmetic — removing arithmetic from a memory-bound kernel does not save time. A secondary pre-registered training study establishes that the communicating architecture improves return; its six sparsification contrasts are under-powered nulls at five seeds. Constructively: a successor method must economise on the selector, not the aggregation. We release the harness, the corrected implementation, and a checker that re-derives every headline number from raw artifacts.

1 Introduction

Cooperative multi-agent reinforcement learning systems increasingly pass learned messages between agents, and attention is a widely used mechanism for deciding what to listen to (Das et al., 2019; Jiang & Lu, 2018; Jiang et al., 2020). Because every agent attends to every other, the channel costs $\mathcal{O}(N^2)$ per step, and a natural proposal follows: restrict each agent to its top-k peers by attention score, with $k \ll N$. The efficiency case is easy to state — if dense aggregation touches $N-1$ peers and sparse aggregation touches k, a $1-k/(N-1)$ fraction of the work disappears — and easy to accept, because it is an operation count and operation counts are cheap to compute and hard to argue with.

This paper is a report on why we abandoned that design. We set out to build *score-then-select* top-k communication for multi-agent RL — forming the complete score vector for each agent, then keeping its k largest entries — on exactly the rationale above. Within the set of multi-agent communication papers we read for this work and audit in Section 2, none uses a score-then-select rule — selecting peers by attention score after computing all of them; the restriction mechanisms there are largely structural, and, as we show

below, structural selection is not subject to the limit we derive. We state that as a property of an audited set rather than of the literature, since an absence claim over a literature would need a search protocol we did not run. The argument we examine here is therefore one we made ourselves, and the implementation errors we report are our own. We think the result is worth reporting anyway, because the reasoning that led us there is standard, the failure has three distinct causes at three levels of abstraction, and only the third is specific to our hardware.

The count is smaller than it looks. In the score-then-select setting studied here, selecting the top-k peers requires first forming the complete score vector. The score matrix is therefore irreducible, and only the value aggregation — one of the two equal multiply–accumulate terms in an attention operation count — can be sparsified at all. Section 3 states this as a proposition: the achievable reduction is bounded above by $2\times$, for every k and every N, approached only as $k/N \to 0$. The familiar $1 - k/(N-1)$ describes one of those two terms. This is a bound on a class of rules, not on sparsity in general, and we are careful about which: content-dependent mechanisms that identify candidates *without* materialising the full pairwise score matrix — nearest-neighbour retrieval, clustering and routing, indexing, hashing — fall outside it. The escape route is therefore precise, and we make it explicit because it is the design target for any successor method: if the retained set is fixed before the scores exist, the score matmul becomes sparsifiable too and the ratio is no longer bounded by 2. Charging C_{sel} for the selector, the count ratio is asymptotic to N/k when $C_{\text{sel}} = o(Nkd)$, and equals N/k only when the selector is free — which a fixed or precomputed pattern is, and a hash or a router is not automatically.

Even the reduced count need not be executed. A count is a model of cost, not a measurement of it, and an implementation can fail to perform the saving the count describes. Applying top-k as a *mask* on the dense attention matrix materialises the discarded entries as zeros and runs the same dense $(B, N, N) \times (B, N, d)$ aggregation matmul, so the executed dense aggregation MAC count is unchanged and selection and masking are added on top. This is what our own first implementation did, and a FLOP model based on $1 - k/(N-1)$ credits it with a saving that grows analytically as $k/(N-1)$ falls: 75% at our headline N=512, k=127 cell. We implement the genuinely sparse gather instead, verify it computes the same function to floating-point tolerance while executing measurably fewer FLOPs, and benchmark dense, masked and gathered as three separate curves against a fused dense baseline.

And the executed saving need not be time. The gathered implementation is slower than unfused dense in all 46 cells of a head-width $\times$ batch $\times$ N sweep, and slower than a fused kernel in 45 of them. Section 4 decomposes why: the sparse contraction is itself slower than the dense matmul it replaces, because a batched matrix–vector product has far lower arithmetic intensity than a real GEMM; selection and gather move substantial memory for no arithmetic; and at N=512 the sparse path moves $3.8\times$ the traffic for $0.6\times$ the arithmetic. Removing arithmetic from a memory-bound kernel does not make it faster. We name the dense baseline and the envelope with every number, because the two baselines differ by up to $4.6\times$ and which one is meant sometimes changes the answer.

Contributions.

- **Analytical.** A $2\times$ ceiling on the operation-count saving achievable by any *score-then-select* rule — one that fixes the retained set only after the complete score vector exists — with its assumptions separated, its domain bounded away from retrieval- and routing-style selection, and its escape route stated as a constructive proposition that charges for the selector (Section 3).
- **Implementation.** A masked/gathered/fused taxonomy of the same function, a corrected sparse implementation, and regression tests pinning that the two top-k variants agree numerically while executing different work (Sections 3, 4, Appendix A.5).
- **Systems evidence.** A benchmark campaign over $N \leq 512$, head width 16–256, batch 32–4096, two dtypes, three execution modes and two architectures, with a component decomposition, a roofline account of the cause, and a drift control (Section 4).

- **Learning evidence, bounded by its power.** A study whose analysis plan was registered after 8 of 40 runs had completed and before their outcomes were inspected, separating "the communication machinery does nothing" from "sparsifying it is free", with positive controls that pass — for the architecture, not the channel in isolation — and six sparsification contrasts reported as the underpowered nulls they are (Section 5).

- **Methodological artifacts.** Seven reproducible benchmarking pitfalls, each of which produced a spurious result here, and a released harness with a checker that re-derives every headline quantity from the raw JSON and fails on any disagreement (Section 6).

Every claim concerns centralised-inference computational cost on one device, not bandwidth-constrained distributed deployment, where the objective is to reduce messages physically transmitted and where top-k sparsity may well be the right design. Section 7 states the full envelope, including where our own implementation is known to leave performance on the table.

2 Related Work and a Reporting Audit

2.1 Attention-based inter-agent communication

CommNet (Sukhbaatar et al., 2016) broadcasts mean-pooled hidden states; TarMAC (Das et al., 2019) introduces signature-based targeting so that receivers weight senders; IC3Net (Singh et al., 2019) gates whether an agent speaks at all; ATOC (Jiang & Lu, 2018) forms bounded communication groups; DGN (Jiang et al., 2020) restricts attention to a spatial neighbourhood; MAT (Wen et al., 2022) casts the joint policy as a sequence model. The pattern persists in applied systems: MARL-IoTP (Hamdan et al., 2026) pairs heterogeneous MAPPO agents with an attention-based message aggregator for edge orchestration, and ablates that aggregator against mean pooling. Several of these works motivate restricted communication with a cost argument. DGN, for instance, observes that receiving a large amount of information "requires high bandwidth and incurs high computational complexity" (Jiang et al., 2020) — a claim about both transmitted bytes and local compute, of which only the second is what this paper measures.

Two points about this group matter for what follows. First, the restriction mechanisms are mostly *structural*: DGN's neighbourhood is a fixed spatial prior, so it falls squarely under Proposition 2. ATOC is the more nuanced case — its groups are formed by a learned attention unit that decides *whether* an agent should initiate communication, so it is not a fixed pattern, but the grouping is settled before the communication attention weights are computed, so its cost enters as the selector term of Proposition 2 rather than as a full score matrix. Neither mechanism is the score-then-select rule that Proposition 1 constrains.

Second, we did not find a method in the papers we audit that selects peers by attention *score* after computing all scores, which is the mechanism this paper analyses. We flag that explicitly because it determines what our result does and does not bear on: we are evaluating a design we proposed and built, and the ceiling of Section 3.2 constrains that design, not the structural selectors above.

One proposal outside that audited set is the closest published relative of the mechanism we examine: the adaptive sparse communication graph of Sun et al. (2020), which learns which peers to attend to by replacing softmax with a generalisation of sparsemax, so that the normalisation itself emits exact zeros. It motivates that design partly through scalability — the paper states that the resulting sparsity "can reduce the complexity of both the forward pass and the back-propagation of the policy and value networks" — and it explicitly considers, and rejects, hard top-k thresholding as non-differentiable. Two things follow. First, the design we set out to build is a natural one to reach for here. Second, because the sparsity-inducing map is applied to raw attention logits already formed for every agent pair, the selection is *post-score*: under our accounting it removes aggregation work but not the score-computation term, so it sits inside the class Proposition 1 constrains. That is a statement about where the method lands in our accounting, not a contradiction of its claims — its reported gains are in sample efficiency at up to 30 agents, it reports no measured runtime, and the systems question we ask is not one it asks.

2.2 Efficient and sparse attention

The efficient-attention literature is where the operation-count argument for sparsity is most developed, and it separates into mechanisms our two propositions treat differently. *Predetermined patterns*: Longformer (Beltagy et al., 2020) and BigBird (Zaheer et al., 2020) fix which pairs interact in advance — sliding windows, global tokens, random edges — so the score computation itself becomes sparse and the saving is asymptotic; their selector is essentially free, which is case (i) of Proposition 2. *Routed or hashed candidate sets*: Reformer (Kitaev et al., 2020) restricts candidate interactions by a locality-sensitive hash, and Routing Transformer (Roy et al., 2021) by online k-means clustering of queries and keys. Both are content-dependent, both evaluate scores only for candidate pairs, and together they are why Proposition 2 carries a selector term at all: hashing, clustering and bucketing are cheaper than the score matrix but they are not free. *Kernelised attention*: Performer (Choromanski et al., 2021) does not select a subset of interactions in this sense at all — it avoids materialising the quadratic score matrix through a kernel factorisation that approximates the softmax attention map. The first two groups escape Proposition 1 because they decide what to keep without forming the complete score matrix; Performer falls outside the proposition's premise, since it keeps every interaction and reorders the algebra instead.

Top-k attention in the sequence setting is the closest external analogue of the mechanism we build, and it makes a distinction we rely on. Gupta et al. (2021) process queries in chunks and, for each query, compute its k largest scores against all keys *exactly* — a score-then-select rule in our sense. Their stated benefit is memory: chunking reduces peak attention memory from $\Omega(L_Q L_K)$ to $O(C L_K)$ for chunk size C while remaining a drop-in replacement for vanilla attention in pretrained models without corrective pre-training, and they describe chunk size as trading maximum memory against the slowdown from processing chunks sequentially. Memory efficiency, operation-count efficiency and wall-clock efficiency are three different quantities, and a mechanism can buy one while paying in another. Ours is a question about the second and the third, in the substantially smaller agent-count regime of multi-agent communication.

Sparsemax (Martins & Astudillo, 2016) and α-entmax (Peters et al., 2019) obtain exact zeros from the normalisation rather than a hard cutoff — the family Sun et al. (2020) generalises for multi-agent communication. Those papers make no claim about attention cost; the observation that their zeros arrive *after* the scores exist, and that they are therefore subject to our ceiling, is our inference rather than theirs. Gumbel-softmax (Jang et al., 2017) provides the relaxation used by learned discrete gates. Constrained-RL formulations (Achiam et al., 2017; Stooke et al., 2020) supply general machinery for optimising under a constraint, which we adapt to a communication budget rather than trading the budget against return at a fixed weight. We implement entmax and a Lagrangian budget as arms in Section 5.

2.3 Sparse routing and mixture-of-experts

Score-then-select top-k is used at scale in sparse mixture-of-experts, where a router scores every expert and dispatches each token to the best k (Shazeer et al., 2017; Fedus et al., 2022). The structure of the argument is the same one we examine, and so is the exemption: these architectures are designed so that the router is small relative to the expert computation it gates, so the analogue of our $N^2 d$ score term is a minor cost rather than an equal one, and the saving is real. That contrast is instructive. Whether score-then-select pays depends on the ratio between the cost of scoring all candidates and the cost of the work being skipped — large in MoE, and equal to one in single-head attention, which is exactly Proposition 1.

2.4 Measuring efficiency rather than counting it

That operation counts, memory traffic, kernel fusion and realised latency are distinct quantities is established in the systems literature and in parts of the attention literature. The roofline model (Williams et al., 2009) formalises when arithmetic ceases to govern runtime. FlashAttention (Dao et al., 2022) and memory-efficient attention (Rabe & Staats, 2021) make the point from the other direction: the winning implementations of dense attention reduce memory traffic without changing the operation count at all, which is precisely the axis a FLOPs-based argument cannot see.

One precedent anticipates the distinction this paper turns on. Longformer reports runtime and memory for *three* implementations of the same attention pattern — a loop version, a vectorised "chunks" version and a custom CUDA kernel — and notes that the chunked variant consumes roughly twice the optimal memory because it computes some zero values. In our terms, which its authors do not use, that is an over-computing vectorised implementation measured separately from a genuinely sparse one, with the difference reported.

A reporting audit. Motivated by that precedent, we checked whether efficiency claims in the papers we drew on are accompanied by a measurement. For each paper we read the full text, including appendices and the official supplement where one exists, searched it for seven temporal keywords, and recorded whether it makes a computational-efficiency or complexity claim about its mechanism, whether it reports any measured temporal quantity attributable to that mechanism, and the figure, table or sentence that settled the decision. Appendix A.10 states the protocol and its one scope caveat, and the completed sheet is released, because a "no" is an absence claim and an absence claim is only as good as the search behind it. Table 9 reports the result.

Among the five multi-agent papers making an efficiency claim, one reports a measured temporal quantity for its mechanism; among the six efficient-attention and mixture-of-experts papers, five do. We draw no conclusion about either field from twelve papers. The asymmetry is worth stating only because it describes the evidence base we ourselves worked from when we adopted the operation-count argument, and because the argument does not transfer cleanly in any case: the efficient-attention results are obtained at sequence lengths of 10^3–10^5, while multi-agent communication operates at $N \lesssim 10^2$, so the hardware bottleneck the two regimes face need not be the same one — which is precisely what Section 4 measures.

Where sparse attention does pay. Recent systems work supplies the other side of that ledger, and we state it plainly because it bounds our conclusion. Native Sparse Attention (Yuan et al., 2025) ranks compressed key–value blocks and attends within only the selected blocks, with kernels written for the resulting access pattern; it reports up to 9.0× forward, 6.0× backward and 11.6× decoding speedups over full attention at 64k context. MiniMax Sparse Attention (Lai et al., 2026) co-designs blockwise selection with its GPU execution path, including an exp-free top-k selection kernel, and reports 14.2× prefill and 7.6× decoding wall-clock speedups at 1M context on H800. We take these at face value; they sharpen our scope rather than contradict it. Whether sparsity produces wall-clock savings depends on the ratio between the work avoided and the arithmetic and memory cost of selecting and executing the sparse pattern. At 10^5–10^6 tokens the skipped computation is enormous, selection is amortised over blocks rather than performed per pair, and the sparse path is kernel-co-designed. At $N \lesssim 10^2$ none of those three holds: the avoided work is small, per-pair selection dominates, and the sparse path is composed from general-purpose operators. Our claim is about that regime, and what we identify is what a successor design has to change — which is, in effect, what these systems changed.

Empirical rigour in MARL. Our training protocol follows the recommendations of Papoudakis et al. (2021) for systematic MARL benchmarking. Reporting effect sizes and intervals rather than bare means is a rule we add on top of those recommendations. Our environments come from the PettingZoo/MPE family (Terry et al., 2021), and our learner is MAPPO (Yu et al., 2022).

3 What Sparsifying Attention Can and Cannot Save

3.1 Setting and three implementations of one function

Consider one round of scaled dot-product attention over N agents, per batch element, with head width d. Agent i forms a query q_i, and every agent contributes a key and a value; self-attention is masked. Write $s_i = q_i K^\top \in \mathbb{R}^N$ for the score vector and $a_i = \mathrm{softmax}(s_i)$ for the attention weights. Dense attention computes $m_i = a_i V$.

A top-k variant keeps only the k largest entries of a_i and renormalises. We call a mechanism of this kind *score-then-select*: for each query it first forms the complete score vector $s_i \in \mathbb{R}^N$ and only then selects the k largest entries. The term is load-bearing throughout. Every analytic claim below is a claim about that class,

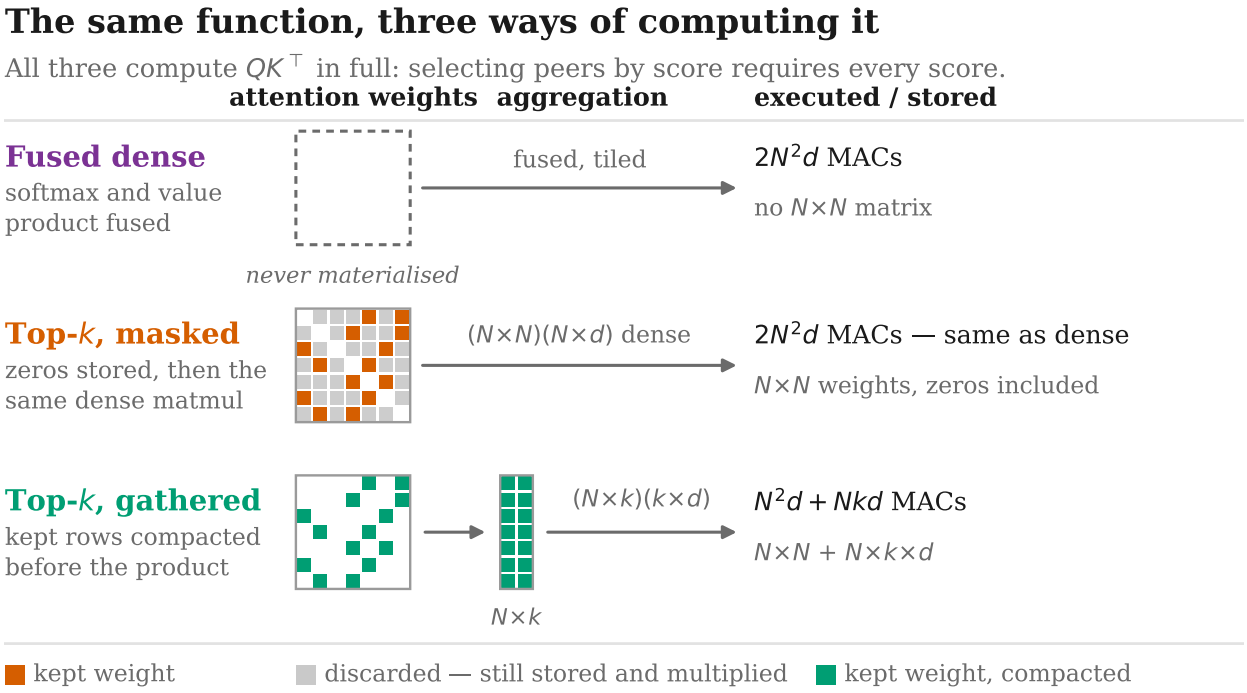


Figure 1: The same top-k attention, computed three ways. All three evaluate every pairwise query–key interaction, because a score-then-select rule cannot rank what it has not scored. They differ in what happens next: the fused kernel evaluates them tilewise and never materialises the full $N \times N$ weight matrix; the masked variant stores the discarded entries as zeros and multiplies them anyway, so its executed aggregation MAC count equals dense's; only the gathered variant compacts the kept rows and shrinks the product. The efficiency argument examined in this paper is an argument about the third row.

not about content-dependent selection in general, and Section 3.2 draws the boundary explicitly. The same function admits implementations with very different costs, and the distinction between them is the one this paper turns on.

Masked top-k. Compute a_i in full, multiply by a $\{0, 1\}$ mask, renormalise, and evaluate $m_i = \tilde{a}_i V$ with $\tilde{a}_i$ still dense in memory. The discarded entries are materialised as zeros, so the executed dense aggregation MAC count is unchanged from dense; selection, masking and renormalisation are added on top of it.

Gathered top-k. Take the top-k of the raw scores, softmax over those k values only, gather the corresponding value rows into a (B, N, k, d) tensor, and contract $(B\cdot N, 1, k) \times (B\cdot N, k, d)$. No zeros are materialised and the aggregation genuinely shrinks.

Fused dense. Compute the same function as dense through a fused kernel (Dao et al., 2022; Rabe & Staats, 2021) that never materialises the (B, N, N) weight matrix.

Figure 1 sets the three side by side. Gathered and masked top-k compute the same function: softmax is monotone within a row, so the top-k weights are the top-k scores, and a softmax over just those k scores equals the dense softmax restricted to them and renormalised — verified to $< 10^{-10}$ in `float64` and pinned by a regression test (Appendix A.5). The operation-count argument is about the gathered variant; timing the masked variant measures an implementation artifact, and timing either against unfused dense alone ignores what a production kernel costs. We report all three throughout, each against both dense baselines.

3.2 A ceiling on what sparsity can save

Analytic counts here are multiply–accumulates (MACs). Under the convention that a multiply–add is two FLOPs every term below doubles and every ratio is unchanged, so the ceiling is the same number either way; we say MACs where the count is analytic and FLOPs only where a counter attached to the running program produced the figure. Counting the two matrix products that constitute attention, and excluding the shared input and output projections:

$$C_{\text{scores}} = N^2 d, \quad C_{\text{agg}} = N^2 d, \quad C_{\text{dense}} = 2N^2 d, \qquad C_{\text{top-}k} = \underbrace{N^2 d}_{\text{all scores}} + \underbrace{Nkd}_{\text{aggregate } k}. \tag{1}$$

Proposition 1 (Score-then-select ceiling)**.** *Let the retained set be fixed by a score-then-select rule — one whose kept indices are determined only after the complete score vector of each query has been formed. Then*

$$\frac{C_{dense}}{C_{top\text{-}k}} = \frac{2N^2 d}{N^2 d + Nkd} = \frac{2N}{N+k} < 2 \qquad \textit{for all } k \geq 1, \tag{2}$$

with supremum 2 *approached only as* $k/N \to 0$*. Within this class, no choice of* k *or* N *yields more than a* $2\times$ *reduction in attention MACs.*

Proof. $C_{\text{top-}k} > N^2 d = \frac{1}{2} C_{\text{dense}}$ for every $k \geq 1$ since $Nkd > 0$. The ratio decreases monotonically in k and tends to 2 as $k/N \to 0$. □

We use $N^2 d$ rather than $N(N-1)d$ because it is the count the implementations execute: self-masking writes $-\infty$ into the diagonal of a full $N \times N$ score matrix rather than skipping work. Counting the diagonal exactly would give $2(N-1)/\big((N-1)+k\big)$, strictly smaller than $2N/(N+k)$ for every $1 \leq k \leq N-1$, so Proposition 1 is a valid upper bound under either convention and slightly loose under the exact count. The figure $1-k/(N-1)$ describes the value-aggregation term alone; read as *the* saving it overstates the achievable reduction by a factor approaching two, before any question of whether it is realised in time.

The assumptions are worth separating, because only one of them is load-bearing. **(A1) Single round**: with R rounds both terms scale by R and the ratio is unchanged. **(A2) Full-score selection**: the retained set is determined only after the complete candidate score vector has been computed, so all N scores must be formed. **(A3) Per head**: with H heads of width d/H both terms scale identically. **(A4) Projections excluded**: including the shared qkv and output projections moves the ratio strictly closer to 1, so the bound remains valid. **(A5) Selection cost excluded**: counting the top-k operation itself can only decrease the ratio, and Section 4.4 shows this term is large in practice.

A2 is the assumption that carries the result, and it is worth being exact about what it does and does not cover, because the proposition is only as useful as its domain is precise. A2 is a statement about *when* the retained set is fixed — after the full score vector exists — and not about whether the rule depends on the data. Mechanisms that are content-dependent but not score-then-select are outside the class and are not constrained by Proposition 1: exact or approximate nearest-neighbour retrieval, clustering and routing, indexing, and hashing can all identify candidates without materialising the complete pairwise score matrix. Routing Transformer's online k-means routing (Roy et al., 2021) and Reformer's locality-sensitive hash (Kitaev et al., 2020) are content-dependent in every ordinary sense and still evaluate scores only for candidate pairs; a hierarchical rule that ranks compressed blocks and then scores only the selected ones (Yuan et al., 2025) is likewise outside A2. What such rules pay instead is a selector cost, and Proposition 2 is what charges them for it. When A2 fails, the ceiling lifts.

Proposition 2 (Escape)**.** *Suppose the kept set is determined* without *forming the complete score vector — structurally, or by a retrieval, routing, clustering, indexing or hashing rule — at total selector cost* $C_{\text{sel}}(N,k,d)$*, and that attention scores are then evaluated only for the* Nk *retained pairs. (We keep the subscript* struct *below for the structural case that motivates it; the algebra is the same for any pre-score selector.) Then*

$$C_{\text{struct}} = 2Nkd + C_{\text{sel}}, \qquad \frac{C_{dense}}{C_{\text{struct}}} = \frac{2N^2 d}{2Nkd + C_{\text{sel}}}. \tag{3}$$

(i) *If* $C_{\text{sel}} = o(Nkd)$ *— in particular for a fixed or precomputed pattern, where* $C_{\text{sel}} \approx 0$ *— the ratio is* asymptotic *to* N/k*, that is* $\big(C_{dense}/C_{\text{struct}}\big)/(N/k) \to 1$*. Exact equality with* N/k *holds only when* $C_{\text{sel}} = 0$*; for every* $C_{\text{sel}} > 0$ *the ratio is strictly smaller.* (ii) *If only* $C_{\text{sel}} = o(N^2 d)$*, the ratio is no longer bounded by* 2*: with* $k/N \to 0$ *it diverges, though not in general at the rate* N/k*.*

Proof. Both statements read off the displayed ratio. For (i), $2Nkd + C_{\text{sel}} = 2Nkd\,(1 + o(1))$, so $C_{\text{dense}}/C_{\text{struct}} = (N/k)\,(1 + o(1))^{-1}$, whose quotient with N/k tends to 1; substituting $C_{\text{sel}} = 0$ gives $2N^2d/2Nkd = N/k$ exactly, and any $C_{\text{sel}} > 0$ strictly enlarges the denominator, so the ratio is then strictly below N/k. For (ii), if $k/N \to 0$ then $2Nkd = o(N^2 d)$, so the whole denominator is $o(N^2 d)$ and the ratio diverges. □

Qualifying rules include a fixed communication topology, a spatial or distance prior, a learned *static* graph, a locality-sensitive hash, and clustering- or retrieval-based routing. The selector-cost term is not decoration: a fixed topology or a precomputed spatial prior really is free at inference and lands in case (i), whereas a hash must be evaluated and its buckets assembled (Kitaev et al., 2020) and a routing rule must maintain and query its clusters (Roy et al., 2021), so their C_{sel} has to be counted rather than assumed away. Case (i) is why Longformer and BigBird obtain asymptotic savings where score-then-select top-k cannot (Beltagy et al., 2020; Zaheer et al., 2020): their patterns are fixed before any score is formed, so nothing is spent deciding. Either case is a concrete design target for a successor method — to exceed 2×, a method must decide where to look before looking, and spend less doing so than it saves. In an environment with spatial structure a distance-calibrated selector is the natural candidate.

Proposition 2 concerns operation counts only. Selection that does not require the complete score vector removes the fixed $< 2\times$ ceiling; when $C_{\text{sel}} = o(Nkd)$ the achievable count ratio is asymptotic to N/k, and exact N/k is recovered only when the selector is free. It does not follow that any particular such method attains a proportional *speedup*, which is a separate question and one the rest of this paper suggests should be asked separately.

3.3 Why the ceiling is not the binding constraint

Proposition 1 bounds a quantity that, in this regime, does not govern runtime. For memory-bound kernels time tracks bytes moved rather than arithmetic performed (Williams et al., 2009), and the two paths differ far more in bytes than in FLOPs.

We compute arithmetic intensity AI = FLOPs/bytes analytically per stage, counting each tensor read and written once, and compare it against a *measured* machine balance: a large GEMM gives achievable FLOP/s, a large device-to-device copy gives achievable bandwidth, and their ratio is the ridge point above which a kernel is compute-bound. Two caveats bound its use. Counting each tensor once ignores cache residency, so at small N — where the working set fits in the GPU's 128 MB L2 — the analytic byte counts overstate traffic and the model does not apply. And our CPU bandwidth probe uses buffers smaller than the host's 96 MiB L3, so the CPU ridge is a cache-bandwidth figure that does not reproduce to better than a factor of a few: repeat measurements give 1.35 and 3.68 FLOP/byte, and a DRAM-resident probe lower still. We therefore treat it as a range and use it only where the conclusion holds across the whole range. Two such statements carry the CPU argument: the gathered path's intensity of ≈ 0.66 lies below every estimate, and the dense path's saturating intensity of $(4d{+}5)/16 = 4.31$ above every estimate. The finite-N dense intensities in between are not stable in that sense, and we do not lean on them.

Figure 2 gives the result, and Table 4 in Appendix A.1 the numbers behind it. The gathered path adds two stages that move substantial memory for no arithmetic — the top-k selection and the gather — so its intensity is pinned near 0.66 and does not improve with N. Dense intensity, by contrast, rises towards $(4d{+}5)/16 = 4.31$. At $N{=}512$ the sparse path moves 4.16 GB to perform 2.76 GFLOP, against dense's 1.11 GB for 4.63 GFLOP: 3.8× the traffic for 0.6× the arithmetic.

Where the model's assumptions hold it predicts well. At $N \in \{192, 512\}$, where the working set exceeds L2 and traffic is real, both paths attain a substantial fraction of achievable bandwidth and the ratio of analytic

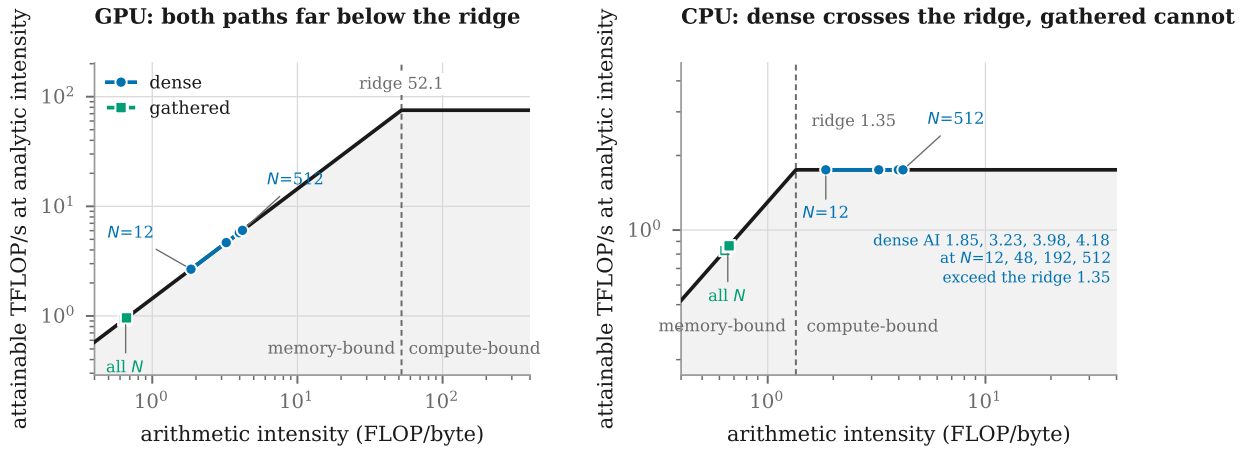


Figure 2: Roofline with measured machine balance. Marker heights are the roof evaluated at each path's analytic intensity, not achieved throughput. On the GPU (ridge 52.1) both paths lie deep in the memory-bound region at every N. On the CPU, whose balance is more than an order of magnitude lower and is plotted at the 1.35 FLOP/byte end of the range we measure for it, the dense path's intensity saturates at $(4d{+}5)/16 = 4.31$ and exceeds that estimate at every N; against the higher 3.68 estimate it does so only from $N{=}192$. The gathered path stays pinned near 0.66 and remains memory-bound under every estimate.

bytes tracks the ratio of measured latencies. At $N \leq 48$ it does not apply, and Section 4.2 shows those cells are governed by kernel launch count instead.

The quantity this analysis bounds is a ratio of intensities — roughly 0.16 at large N — a property of the two algorithms rather than of any device constant, so the qualitative disadvantage should be expected wherever the gathered path stays below the relevant machine balance. That is not itself evidence that it does: an argument constructed to be device-independent cannot demonstrate device-independence. Section 4.6 supplies a second architecture, and Section 7 states what it establishes.

4 Measuring the Cost

4.1 Protocol

We time the communication module in isolation, with the training loop removed. Several provisions below were forced by defects that produced spurious results first (Section 6). Timing uses on-device CUDA events with an explicit synchronisation on the correct device, and any diagnostic calling `.item()` is disabled, because such a call synchronises the device *inside* the timed forward pass. Arms are timed interleaved repetition by repetition rather than in sequential per-arm blocks, so that interference lands on all arms at once and their ratios remain paired; the sequential design produced swings above 100% between identical configurations on this shared host, which interleaving reduced to typically under 2%. We report min over samples as the primary statistic, on the modelling assumption that interference adds time rather than removing it; Appendix A.2 gives median, IQR, inter-run spread and the sensitivity of every headline ratio to that choice. GPU clocks could not be locked — the account lacks the privilege. Executed FLOPs are counted with `torch.utils.flop_counter`, so latency is always reported beside the arithmetic actually performed.

Unless stated otherwise: PyTorch 2.11 / CUDA 12.8, `fp32`, input embedding width 64, head width $d{=}16$, batch 256, $k = 25\%$ of peers, and an idle device.

Table 1: Forward latency (ms, min over interleaved samples) of three implementations of the same function, plus the fused dense baseline, on one NVIDIA Blackwell GPU in `fp32` at batch 256, d=16. The gathered path executes fewer FLOPs than unfused dense at every N and is slower than both dense baselines at every N. Ratios are gathered divided by each dense baseline.

		dense		**top-k**		**ratio**		**MFLOP**
N	k	fused	unfused	MASKED	GATHERED	/unfused	/fused	gath. vs dense
6	1	0.120	0.127	0.175	0.146	1.15	1.22	10.6 vs 10.8
12	2	0.121	0.127	0.175	0.148	1.16	1.22	21.8 vs 22.8
24	5	0.102	0.123	0.171	0.144	1.17	1.41	46.6 vs 50.3
48	11	0.110	0.131	0.180	0.151	1.15	1.38	105 vs 120
96	23	0.106	0.127	0.198	0.205	1.61	1.93	257 vs 315
192	47	0.151	0.206	0.503	0.596	2.90	3.95	703 vs 931
384	95	0.290	1.157	3.149	2.844	2.46	9.82	2161 vs 3070
512	127	0.431	1.991	7.057	6.534	3.28	15.18	3553 vs 5167

4.2 Three curves

Table 1 is the central measurement. The gathered implementation does execute less arithmetic than unfused dense — 3553 against 5167 MFLOP at N=512 as the counter reports them, a 31% reduction consistent with Proposition 1 — and it is nonetheless 3.28× slower than unfused dense and 15.18× slower than the fused kernel. Correcting the implementation helps without being sufficient: replacing the masked variant with the gathered one improves the ratio against unfused dense from $\approx 1.40\times$ to $\approx 1.16\times$ for $N \leq 48$. The improvement does not persist — at N=96 and N=192 the gathered path is slower than the masked one it corrects (0.205 vs 0.198; 0.596 vs 0.503). The explanation we offer for that reversal is the roofline account of Section 3.3 — the gather's traffic grows faster than the masked path's wasted arithmetic — and it is an inference from the byte accounting in Table 4, not a separate controlled experiment.

The two ends of this table are governed by different mechanisms, which bounds how far the numbers generalise. For $N \leq 96$ the unfused dense arm is nearly flat at 0.12–0.13 ms while its work grows 256-fold: those cells are bound by kernel launch count rather than bandwidth or arithmetic, and the 1.15–1.17× ratios there reflect how many kernels each arm issues, which makes them sensitive to how many arms share a timing group — two of our own sweeps disagree by 5× on one such cell (Section 6, P7). From N=192 upward the working set exceeds L2, traffic is real, and the roofline account of Section 3.3 applies.

Robustness. The ordering is unchanged under `torch.compile` and CUDA-graph capture: at N=512 the ratio against unfused dense is 3.29, 3.30 and 3.31 for eager, compiled and graph-captured execution respectively. It is also unchanged at batch 4096, where the largest cells become FLOP-bound. In `bf16` the gap is wider, reaching 19.7× the fused kernel at N=512. FlashAttention serves no arm here — our fused baseline passes a self-attention mask, which this build rejects for FlashAttention in every mask form, and `fp32` excludes it again on dtype — so the fused arm runs PyTorch's memory-efficient CUTLASS kernel, read from the profiler rather than asserted. The substitution runs against us: forcing FlashAttention on the same unmasked `bf16` shape is 1.5–2.5× faster than the kernel we do get, so a baseline able to reach it would widen the gap we report. Appendix A.5 gives the probe and its numbers.

Selection is tuned, and it matters how much. The gathered path calls `torch.topk` with `sorted=False`. The ordering of the k retained entries is irrelevant here — the softmax and the gather are permuted consistently by the same indices, and we verify the two calls agree to 5.4×10^{-7} — while the unsorted call was cheaper in every shape we tested under PyTorch 2.11. The difference is not small on the stage that dominates this path: with the default `sorted=True` the selection stage costs up to 35% more and the whole gathered arm 5–24% more, which at N=192 is the difference between a ratio of 2.90 and one near 3.8. We report the tuned configuration throughout, because the question is whether a tuned sparse implementation wins, not whether a careless one loses. The size of that gap is itself part of the finding: a

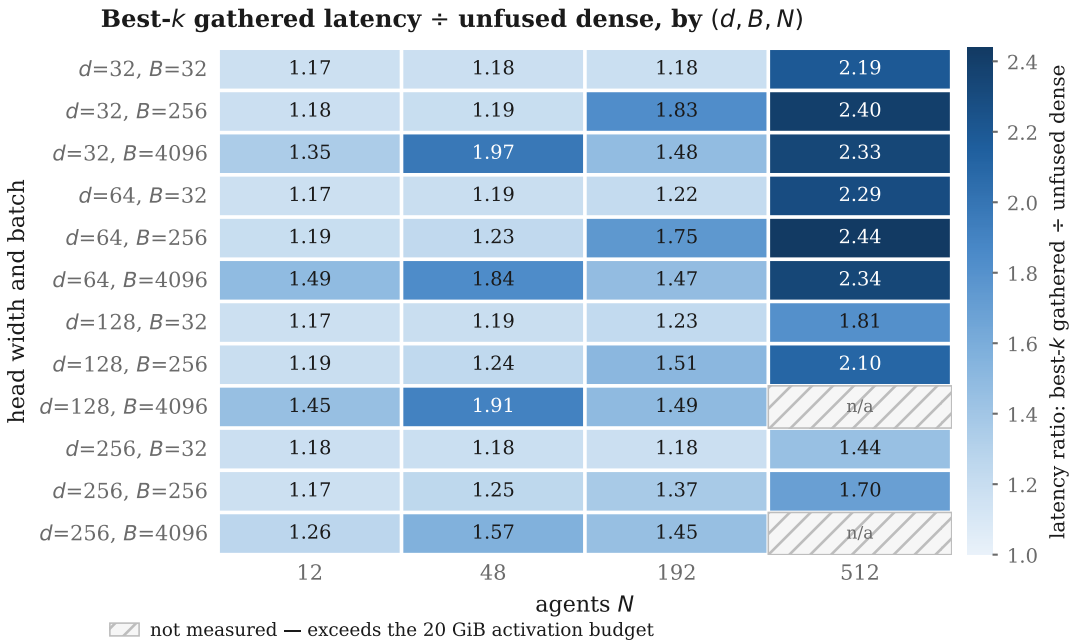


Figure 3: Crossover surface: best-k gathered latency divided by *unfused* dense, on GPU in `fp32` eager mode. Every one of the 46 measured cells exceeds 1. At N=512 the ratio trends downward as head width grows ($2.40 \to 2.44 \to 2.10 \to 1.70$ at B=256: not strictly monotone, since d=64 is slightly worse than d=32), so the surface is flat in batch but not in d; the sweep ends at d=256. Hatched cells exceeded the memory budget. Against the *fused* baseline, one of the 46 cells falls below 1 (Section 4.3).

sparse path has more sites of avoidable implementation overhead than a dense one, and an operation count sees none of them.

4.3 Is there any (d, B, N, k) where sparsity wins?

Reporting isolated favourable cells as anomalies is not a characterisation. We swept head width $d \in \{32, 64, 128, 256\}$ against batch $\in \{32, 256, 4096\}$ and $N \in \{12, 48, 192, 512\}$, taking for each cell the *best* k for the gather path — the most favourable operating point available to it.

Gathering beats unfused dense in 0 of 46 measured cells and beats the fused baseline in 1 of 46 — at d=128, B=4096, N=12, ratio 0.86, where the fused kernel's fixed overhead dominates at small N. The underlying numbers are tabulated in Appendix A.3. These 46 cells are a single sweep: GPU, `fp32`, eager mode. The CPU, `bf16`, compiled and CUDA-graph results are separate and smaller sweeps, and we do not pool them into this count.

Across the surface the ratio stays within 1.17–2.44×. It is flat in batch across a 128-fold range, which suggests the outcome is not an artifact of one operating point. It is *not* flat in head width: at N=512 it trends downward as d grows ($2.19 \to 2.29 \to 1.81 \to 1.44$ at B=32), though not strictly — d=64 is slightly worse than d=32 at both batch sizes we can read the trend from — and the sweep stops at d=256 with the two most favourable remaining cells skipped for memory. We therefore make no claim for $d > 256$ and regard the head-width trend as unresolved. Head widths above 256 are outside the range used in the multi-agent communication systems of Section 2, which is why the sweep was designed as it was.

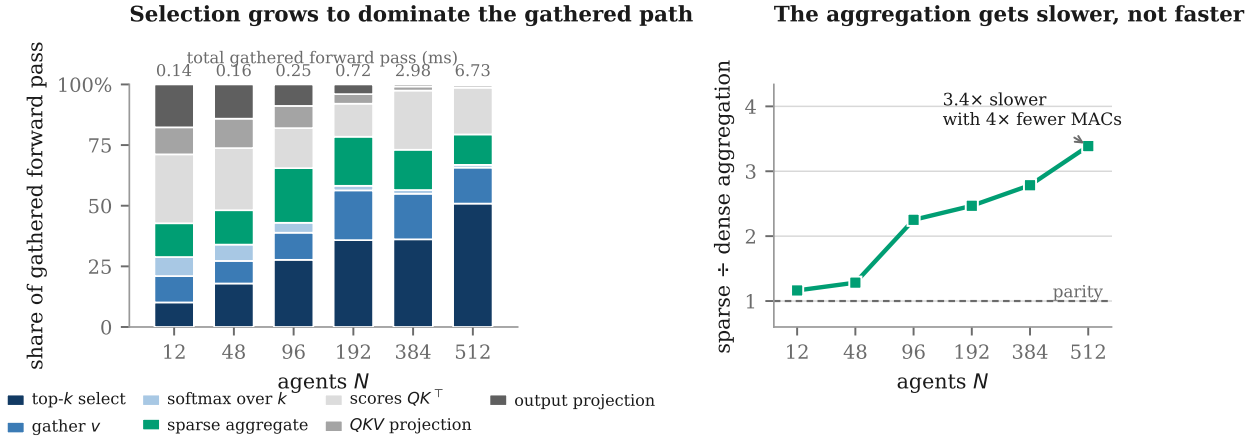


Figure 4: **Left:** how the gathered path spends its time, as a share of its full forward pass (absolute totals above each bar, in ms). Top-k selection grows from 10% to 51% of the budget as N rises, and selection together with the gather and the softmax over k grows from 29% to 67%. **Right:** the aggregation that sparsity was supposed to shrink. The sparse contraction is slower than the dense matmul it replaces at every N, reaching 3.4× at N=512 while performing roughly four times fewer multiply–accumulates.

4.4 Where the time goes

Table 1 says *that* the sparse path loses; the following says where its time is spent. We time each stage in isolation under the same protocol and form two quantities: the *saving* the sparse contraction buys ($\texttt{agg_dense} - \texttt{agg_sparse}$) and the *overhead* selection costs ($\texttt{topk} + \texttt{gather} + \texttt{softmax}_k - \texttt{softmax}_N$).

Figure 4 and Table 7 give two results, both against the hypothesis they were meant to test. First, the aggregation saving is negative: the sparse $(B{\cdot}N, 1, k) \times (B{\cdot}N, k, d)$ contraction is 3.4× slower than the dense $(B, N, N) \times (B, N, d)$ matmul it replaces at N=512, because a batched matrix–vector product has far lower arithmetic intensity than a real GEMM and cannot use the same hardware paths. Second, selection is the largest single stage: `topk` alone costs 13.7× the entire dense aggregation at N=512, and 51% of the gathered path's forward pass.

How much of this a better kernel could remove is a question the roofline model answers: a substantial part of the selection term, but not the outcome. At N=512, B=256 the top-k must read the 268 MB score matrix and write values and indices, about 0.47 GB; at the 1444 GB/s we measure as achievable, moving those bytes and nothing else would take 0.32 ms against the 3.426 ms observed, so PyTorch's top-k runs at roughly 9% *copy-equivalent bandwidth efficiency* on this shape even unsorted. That is a bandwidth-only lower bound and an optimistic one, since a real selection kernel also compares, ranks and writes irregularly; the gather stage, by contrast, already runs near bandwidth. Selection is therefore *not* intrinsically expensive in the way the raw decomposition suggests, and we withdraw any suggestion that it is. What survives is the ordering: substituting even that unattainable copy-equivalent select into the N=512 cell gives roughly 3.4 ms against 1.99 ms unfused and 0.43 ms fused — an optimistic counterfactual that still loses, rather than a proof that no kernel can cross over. The conclusion does not depend on the selection kernel being bad. It depends on the score matrix having to be formed and ranked at all, which is assumption A2.

4.5 The evaluated learned gate costs $\mathcal{O}(BN^3)$ memory

Our per-agent learned gate over $k \in \{1, \dots, N-1\}$ broadcasts a $(1, 1, K, N)$ cumulative-keep tensor against $(B, N, K, 1)$ gate probabilities with $K = N - 1$, materialising $(B, N, N-1, N)$ before reducing. Activation memory is therefore $\mathcal{O}(BN^3)$ against dense attention's $\mathcal{O}(BN^2)$ and the fused kernel's $\mathcal{O}(BNd)$.

Measurement confirms the derivation (Appendix A.6). Fitting peak $\sim N^p$ over $N \geq 48$ with a constant offset removed gives $p = 3.04$ for the gate, against 1.96 for dense, 2.02 for masked, 1.98 for gathered and 1.36

for fused, and at N=384 the closed form predicts 97.5% of the measured peak; the gate consumes 55.2 GiB at N=384, batch 256, and exhausts the device at N=512. This is a result about the implementation we evaluated, not a lower bound on learned gates — the same function admits an $\mathcal{O}(BN^2)$ formulation — so we report it as an easy and costly implementation trap, belonging with the pitfalls of Section 6. Its interest is that the arm whose stated purpose is to reduce cost was the first to exhaust the device.

4.6 Cross-architecture check

Running the same arms on an x86_64 CPU tests whether the ordering depends on GPU kernel selection. It does not, but obtaining a usable measurement required controlling an effect unrelated to attention. With the default glibc allocator the multi-MiB (B, N, N) temporaries each arm allocates and frees per forward are returned to the kernel and re-faulted on the next call: at N=192 this adds $\approx$ 37,000 minor page faults per forward and inflates latency by up to 8.8×. The tax is proportional to the bytes of large temporaries each arm churns, so it differs per arm and does not cancel in a ratio, and in our archived un-pinned run it landed hard enough on the dense arm at N=48 to invert the ordering there. That was an allocator measurement, not an attention measurement. We pin the heap and count minor faults per forward inside the timed region; Appendix A.7 gives the procedure and what the harness records so the before-and-after can be inspected rather than taken on trust.

With the heap pinned, the gathered path is slower than both dense baselines at every N: 1.96–3.13× unfused dense and 2.54–28.9× the fused baseline. The N=48 inversion disappears entirely — gathered is 3.13× slower there once the fault storm is removed, against an apparent 0.81× before — and it was the only cell in this study, on either device, where a gathered path appeared to beat a dense one on equal terms. The fused kernel is the arm least affected by the allocator, because CPU flash-attention never materialises a (B, N, N) tensor.

Two effects are worth separating. The gathered-versus-unfused penalty is of the same order on both devices and at N=192 is smaller on CPU (2.10×) than on GPU (2.90×) — agreement in sign, not magnitude. The much larger CPU figure against the *fused* baseline is dominated by a different effect: the fused kernel's tiling avoids streaming a 37.7 MB score matrix through a 96 MiB L3 shared across two sockets, giving it a 13.8× advantage over unfused dense at N=192 against 4.6× on the GPU. That is evidence about which dense baseline matters, not about arithmetic intensity.

4.7 Drift control

Because clocks are unlocked and the host is shared, we re-ran the identical sweep twice on the same device: once with the device otherwise idle, and once with a concurrent GEMM-and-allocation load running on it.

Table 8 in Appendix A.8 gives the result. With the device idle the ratios reproduce to within 1.4% at every N and absolute latency to within 2.6%. Under concurrent load agreement degrades sharply — absolute latency moves by a median of 68% — while the paired ratios move by a median of 34%. The pairing buys less than that suggests: the *worst* ratio cell moves by 116%, marginally more than the worst absolute-latency cell at 114%, so what pairing reliably removes is the typical drift, not the tail. Headline latency must therefore be measured on an unloaded device, and interleaving plus minimum-over-samples reduces contention sensitivity without eliminating it.

Direction and magnitude should be reported separately. Under load all eight ratio cells moved against the sparse path, by +12% to +116%; on the idle repeat the movements are small and mixed, none exceeding 1.4%. The relevant quantity is therefore the margin. The smallest gathered/unfused-dense ratio we observe on GPU across every run, precision and execution mode is 1.15×, and on CPU 1.96×; the smallest against the fused baseline is 0.89×, at N=6 and batch 4096, where fixed kernel overhead dominates. No condition we measured brings any GPU cell below 1.15× unfused dense, and the one contention regime we could measure moves every cell further from one. We cannot exclude an interference pattern that would penalise the dense path more — a competitor evicting its working set rather than adding launch latency might — but none available to us does. This is a same-device repeat rather than a second card, because the second card was

not free of other tenants when these measurements were taken; it establishes stability across runs and hours, not independence from one physical card.

5 Does Sparsification Cost Return?

Sparsification could still be worthwhile if it cost nothing in return, and testing that requires care: on a task where the channel is not needed, "sparsification is free" and "the channel was never used" make identical predictions. We therefore gate the study on a positive control and pre-register the analysis. The study is secondary to the computational result. Its role is to establish that the architecture being sparsified is doing something, so that the sparsification contrasts are not vacuous, and to report what a five-seed design can and cannot resolve about return; it does not isolate the learned attention channel, for reasons given below.

5.1 Design and pre-registration

Twelve arms, $N \leq 12$, five seeds, 2M environment steps each, in four priority tiers. All N-scaling is delegated to the training-free measurement of Section 4, because end-to-end wall-clock in this pipeline is environment-bound and cannot attribute a latency difference to the module.

We use two communication-relevant positive controls. In `simple_reference` each agent observes only its partner's goal, so the task requires information to pass between agents; the environment provides its own discrete utterance channel, and our control ablates the *learned attention channel* on top of it rather than removing communication altogether. It is fixed at N=2, where attention entropy is identically zero, so it validates the message pathway rather than the selection mechanism. To carry the k arms we introduce a partially observable `simple_spread`: for agent i and peer j, if $\|\text{peer_rel}[i,j]\|_2 > R$ that entry is zeroed, with R=1.0 and nothing else changed. Under a uniform random policy the mean fraction of peers visible is $0.342, 0.307, 0.263, 0.235$ for $N = 3, 6, 9, 12$, while the absolute number visible rises $0.68 \to 1.54 \to 2.10 \to 2.58$: the world does not grow with N, so density rises and a fixed radius hides a different fraction at each N. That is a difficulty confound across N, and it is why no claim in this paper compares returns across N in this environment. Appendix A.12 gives the remaining characterisation, including a visibility aliasing rate below 0.15% and a 7–19% parameter-count advantage for the communicating arms, which means the positive controls establish that the communicating architecture helps rather than isolating the channel.

The analysis plan was committed after 8 of the 40 tier-2–4 runs had completed and before any of their outcomes was inspected: prospectively registered before the tier-2–4 outcomes were unblinded, with data generation already under way, which we do not obscure. The tier-1 arms were complete and had been inspected, which is why the tier-1 gate is excluded from the inferential family. Commit history establishes the ordering; timestamps cannot establish that a file on disk was never opened, so the plan's independence is a claim about our conduct rather than something the artifact proves. Appendix A.9 gives the chronology and the single amendment. The plan fixes the endpoint (final greedy evaluation return, one number per seed), the six contrasts and their direction, the test (exact two-sided permutation on the difference of means over all $\binom{10}{5} = 252$ assignments), the effect sizes, the interval (percentile bootstrap, flagged approximate at n=5), and Holm–Bonferroni correction at FWER 0.05. It also fixes the power expectation in advance: the smallest attainable two-sided p is $2/252 = 0.0079$ against the first Holm threshold of $0.05/6 = 0.0083$. The margin between them is 5×10^{-4}, so any rejection at all requires the smallest raw p to sit at the discrete floor, which for an untied difference of means means one arm's five seeds lying entirely above the other's. Holm is a step-down procedure and its later thresholds are looser, so further rejections remain possible once the first is obtained; what the design forecloses is obtaining the first on anything short of complete separation. Any null is under-powered rather than evidence of absence, and no equivalence claim is available because no TOST margin was pre-registered.

5.2 Results

Table 2 reports the outcome.

Table 2: Tier-1 positive control (inspected before pre-registration; excluded from the correction) and the six pre-registered contrasts. $A - B$ is the difference of means, HL the Hodges–Lehmann estimate, p the exact two-sided permutation p-value and p_{Holm} its Holm–Bonferroni adjustment over the family of six. C1 is the pre-designated primary contrast.

	Contrast	$A - B$	HL	d	p	p_{Holm}
Positive control						
G1	dense − MAPPO, `simple_reference`	+4.26	—	—	0.008	—
G2	dense − MAPPO, PO `simple_spread` N=6	+11.59	—	—	0.008	—
Pre-registered family						
C1	gathered k=2 − random k=2	+4.03	+4.57	+0.60	0.357	1.000
C2	gathered k=2 − dense	−0.61	−1.98	−0.12	0.865	1.000
C3	gathered k=4 − gathered k=1	+1.76	+1.21	+0.26	0.659	1.000
C4	gathered k=3 − dense, N=12	+147.01	+122.94	+1.27	0.064	0.381
C5	Lagrangian budget − dense	−6.74	−3.39	−0.93	0.183	0.730
C6	entmax − dense	−5.56	−5.44	−1.13	0.119	0.595

The communicating architecture helps. Both positive controls separate: +4.26 [+3.15, +5.23] on `simple_reference` and +11.59 [+7.13, +16.44] on the partially observable task. Both attain $p = 2/252$, the floor of a 5 vs 5 permutation test for this statistic on untied data, which here is reached because all five seeds of one arm lie above all five of the other. Measured attention entropy on the N=6 control is 0.35 nats against a $\ln 5 = 1.61$ ceiling, so attention is peaked rather than uniform. What this establishes is that sparsification is being applied to an architecture whose communication machinery improves return under the evaluated setup — the precondition that makes the rest interpretable. It does not establish that the learned attention channel alone accounts for the difference: the communicating arms carry 7–19% more trainable parameters (Appendix A.12), and `simple_reference` supplies an environment utterance channel of its own, as described above. A parameter-matched sham-communication arm would be needed for the channel-specific causal claim, and we do not make it.

The six contrasts do not resolve a difference. All six return Holm-adjusted $p \geq 0.38$, the outcome the pre-registered power calculation anticipated. We report this as a design that could not resolve effects of this size, not as evidence that the effects are absent, and make no equivalence claim. C1 is the contrast the design exists for: `gathered` and `random-`k are matched on k, on executed FLOPs and on every hyperparameter, differing only in whether peers are chosen by attention score or uniformly at random. Its point estimate favours attention (+4.03, HL +4.57, $d = +0.60$) but does not approach significance. The design lacked power to detect a selection effect of that size; nothing follows about whether attention selects informatively. No conclusion in this paper rests on any of the six: they are secondary, pre-registered exploratory contrasts, and the computational result of Sections 3 and 4 stands or falls without them.

Two secondary observations — the uninterpretable N=12 contrast, the bootstrap/permutation disagreement at C4 and C6, and descriptive gate and entropy statistics — are reported in Appendix A.11.

6 Seven Pitfalls in Attention Benchmarking

Each of the following produced a misleading measurement during this study before it was caught. We report them as reusable guidance for anyone benchmarking attention variants, because a benchmarking result is only as trustworthy as the harness that produced it. In every case the symptom was a plausible result rather than an obvious error; Table 3 summarises them.

P1, P3, P6 and P7 are instrumentation errors: they inflate or flatten a measurement and they leave traces — an impossible latency, a non-monotone curve, an unstable repeat. P2 is a design error that lets a shared environment leak into a comparison. P4 and P5 are specification errors about *what is being compared to what*, and they are the dangerous ones, because the resulting numbers are internally consistent, reproducible and stable under every check the other five would fail: they answer a different question from the one being

Table 3: Seven ways our harness measured something other than the algorithm.

Pitfall	**Symptom before diagnosis**	**Corrective practice**
P1. A diagnostic `.item()` inside the timed region	Dense latency nearly flat in N, read as evidence that the $\mathcal{O}(N^2)$ cost does not manifest	Gate diagnostics behind a flag that is off while timing
P2. Timing arms in sequential blocks	Swings above 100% between identical configurations; a 3.7× discrepancy for one arm at one N	Interleave arms repetition by repetition; report paired ratios
P3. `synchronize()` with no device argument	Every kernel at every size reported the event-resolution floor of $\approx 0.4\,\mu$s — a spectacular speedup	Pin the device and pass it to every synchronisation
P4. Benchmarking a masked implementation as sparse	Its aggregation MAC count is unchanged from dense, so the advertised saving is never executed and the "sparse" arm is uniformly slower	Assert executed FLOPs differ, and that the sparse path issues no $(B, N, N) \times (B, N, d)$ product
P5. Comparing only against unfused dense	A 4.6× more favourable comparison at N=512 than against the deployed fused kernel	Report both baselines; check which backend serves the shape and dtype
P6. Trusting the allocator on CPU	Up to 8.8× inflation, a non-monotone curve, and one cell where the ordering inverted	Pin the heap when timing on CPU; check minor faults per forward
P7. Changing the composition of a timing group	Two of our own sweeps disagree by 5× on the same nominal cell	Fix the arm set across sweeps that will be compared, and state it

asked. The defence is to state the operation count each arm executes alongside its latency, which makes a mismatch between the claimed and the executed algorithm visible immediately.

One belief of our own is worth correcting. We assumed that standard counters are blind to fused attention kernels, which would make a FLOPs-only comparison unable to see the baseline that matters. In PyTorch 2.11 that is not so: `torch.utils.flop_counter` reports the fused path correctly, and it was our harness that added an analytic term on top of the counter's output for that arm, publishing a figure 1.83× too large. The narrower lesson generalises better: instrument the counter, then check the counter. A FLOP model based on $1 - k/(N-1)$ will credit code whose executed dense aggregation MAC count is unchanged from dense with a saving of 75% at N=512, k=127, and — as an analytic property of that formula rather than a measured run — with more than 90% wherever $k/(N-1) < 0.1$. Only a counter attached to the running program will say otherwise.

7 Limitations

What the claim covers. Every cost result here concerns centralised-inference computational cost: the wall-clock latency and memory of executing the communication module on a single device, where all agents' observations are available to one process. It is not a claim about bandwidth-constrained distributed deployment, where the objective is to reduce messages physically transmitted and top-k sparsification may well be correct even though it does not reduce local compute. Proposition 1 says nothing about transmitted bytes.

Measurement envelope. Single-round attention; scaled dot-product attention with the fused backends available in PyTorch 2.11 / CUDA 12.8; input embedding width 64; head width $d \in \{16, 32, 64, 128, 256\}$; $N \leq 512$; batch 32–4096; `fp32` and `bf16`; eager, `torch.compile` and CUDA-graph execution; one GPU architecture (NVIDIA Blackwell, sm_120) and one x86_64 CPU. We make no claim outside it. Within it the head-width trend is unresolved: at N=512 the ratio trends downward in d without being strictly monotone, and the sweep ends at d=256, so we do not know where, or whether, it reaches 1 for larger heads.

Our sparse implementation is not optimal, and we can bound by how much. GATHERED is a straightforward PyTorch implementation using `gather` and a batched contraction, not a hand-written fused kernel. We removed one avoidable cost — `torch.topk` sorts by default, worth 5–24% of the arm — and one remains: PyTorch's top-k still runs at roughly 9% copy-equivalent bandwidth efficiency, so a selection-optimal kernel would recover a substantial part of the remaining selection term. Substituting even a copy-equivalent select — optimistic, since a real selection kernel must also compare and rank — still leaves the gathered path near 3.4 ms at N=512 against 1.99 ms unfused and 0.43 ms fused, so the ordering survives; but a reader should treat our ratios as upper bounds on the penalty, and a fully fused selection–gather–contraction kernel remains the most informative single follow-up. It could reduce intermediate traffic, launch overhead and the cost of ranking; it could not avoid forming the score matrix, which is assumption A2. We do not claim that no implementation can ever cross over — only that none available to us does, and that the analytic ceiling bounds what any of them could win.

Generality across architectures. The intensity argument of Section 3.3 predicts the same qualitative disadvantage wherever the gathered path stays below the relevant machine balance, but that argument is device-independent by construction and so cannot itself be evidence of device-independence. Our empirical support is one GPU architecture and one CPU on which the ordering holds with ratios of the same order, plus a same-device drift repeat. We were unable to replicate on a different GPU generation — the older hardware that produced this project's earlier runs is no longer available — so a reader who weights architectural replication heavily should discount the generality accordingly. The CPU comparison against the *fused* baseline is also dominated by cache blocking rather than arithmetic intensity, making it weaker evidence for the intensity mechanism than the unfused comparison is.

Measurement effects, controlled and uncontrolled. Our CPU numbers required pinning the glibc heap; without it, page faults on recycled (B, N, N) temporaries inflate arms unequally and can invert an ordering (Section 4.6). Our GPU ridge point reproduces to within 2%, but the CPU one is a cache-bandwidth figure that does not: repeat measurements give 1.35 and 3.68 FLOP/byte and a DRAM-resident probe lower still, so we use it only where the conclusion holds across that whole range. What remains uncontrolled is that small-N cells are launch-bound and their ratios depend on how many arms share a timing group — two of our own sweeps disagree by 5× on one such cell — so ratios at $N \leq 96$ should be read with that in mind.

Fused baseline and unlocked clocks. Our fused arm passes a self-attention mask, which this build rejects for FlashAttention in any form, so all fused numbers are PyTorch's memory-efficient CUTLASS kernel, read from the profiler rather than inferred; forcing FlashAttention on the same unmasked shape is 1.5–2.5× faster, so a baseline able to reach it would widen the gap we report. We also lacked the privilege to lock GPU clocks, and compensate with interleaved timing, minimum-over-samples reporting, published dispersion (Appendix A.2) and an explicit drift control (Section 4.7). Absolute latency under concurrent load can move by more than 100%; median ratio drift is roughly half the median absolute-latency drift, though the worst loaded ratio cell moves as far as the worst latency cell, so stating claims in ratios buys stability in the typical case rather than in the tail. The drift control is a same-device repeat, so it establishes stability across runs and hours, not independence from one physical card.

Statistical power of the training study. Five seeds per arm gives a minimum attainable two-sided permutation p of 0.0079 against a first Holm threshold of $0.05/6 = 0.0083$, so any Holm rejection requires the smallest family p-value to sit at the discrete permutation floor (Section 5). Resolving the $d \approx 0.6$ effect suggested by C1 would need roughly 45 seeds per arm at two-sided $\alpha = 0.05$ and 80% power, and roughly 70 at the family-corrected $\alpha = 0.05/6$; both figures come from the standard two-sample normal-approximation formula $n = 2\big(z_{1-\alpha/2} + z_{1-\beta}\big)^2/d^2$, which is a guide to order of magnitude rather than a power calculation for the exact permutation test we actually run. We should be direct that this was a design choice rather than a hard constraint: the entire 60-run study consumed 31.9 GPU-hours against a 200-hour guardrail, and a properly powered C1 would have cost about 37.8. We allocated the budget to the measurement campaign instead, which we would do again given that the computational result is the paper's contribution — but the six nulls are a consequence of that allocation, not of an unaffordable experiment, and a reader is entitled to weigh them accordingly.

Environment. The partially observable `simple_spread` variant is our own, documented in Section 5 and released. It carries a difficulty confound across N that we measure and publish, which is why no across-N return comparison appears here; its zeroed visibility mask does not distinguish an absent peer from one at the origin, affecting $\leq 0.15\%$ of visible peers; and at N=12 neither trained arm beats a uniform-random policy, so no contrast at that agent count is interpretable. Results on MPE tasks may not transfer to environments with different observation structure, and our positive controls establish that the communicating architecture helps rather than isolating the channel, since the communicating arms carry 7–19% more parameters.

8 Conclusion

We set out to make inter-agent attention cheaper by restricting each agent to its top-k peers, and the efficiency case for doing so rested on an operation count. Examining that count, and then measuring what runs, produced three separate negative results at three levels of abstraction.

The count is smaller than it appears: because a score-then-select rule must form every score before it can rank them, only the value-aggregation product is sparsifiable — one of the two equal multiply–accumulate terms — and the achievable reduction is bounded by 2× rather than by $1 - k/(N - 1)$. The count need not be executed: masking the dense attention matrix leaves the dense aggregation matmul in place, so an implementation written that way executes the same dense aggregation MACs as dense attention, plus selection and masking overhead, while a FLOP model reports a large saving. And the executed saving need not be time: a genuinely sparse gather, which we verified computes the same function while executing fewer operations, was slower than unfused dense in all 46 cells of our GPU sweep and slower than a fused kernel in 45 of them. Selection and gather move memory for no arithmetic, the sparse contraction is slower than the dense one it replaces, and at N=512 the sparse path moves 3.8× the traffic for 0.6× the arithmetic.

What does not follow is as important as what does. This is not evidence that sparse multi-agent communication is a bad idea. The result is confined to centralised inference on a single device; in bandwidth-constrained distributed settings the objective is the number of messages transmitted, which we neither analyse nor measure. Nor have we shown that no implementation can cross over: we removed one slack in ours — an unnecessary sort, worth 5–24% — and quantified the one that remains, a selection kernel running at roughly 9% copy-equivalent bandwidth efficiency whose bandwidth-only counterfactual narrows the margin without reversing it. What survives every correction we could make is the requirement to form and rank the full score matrix, which is the assumption the ceiling rests on.

The constructive half of the result is Proposition 2. A method that fixes what it will attend to *without* forming the complete score vector — by topology, by a spatial prior, by a hash, by clustering or retrieval — is not subject to the ceiling, and its count ratio is asymptotic to N/k when the cost of running its selector is $o(Nkd)$; exact N/k is recovered only when that selector is free. That condition is satisfied outright by a fixed or precomputed pattern and has to be checked for anything learned, hashed or routed; it is the term a successor method should budget for first. This is not a hypothetical class: it is what the structural selectors in the multi-agent literature already do, and what makes the asymptotic savings in Longformer and BigBird real. In an environment with spatial structure, distance-calibrated routing is the natural instance. A fused kernel that eliminates the intermediate traffic between selection, gather and contraction is the other direction worth pursuing, and would settle how much of our measured penalty is implementation and how much is inherent.

The methodological lesson is not specific to multi-agent RL. An operation count is a model of cost, and like any model it can fail in two ways: it can mis-estimate the quantity it describes, and that quantity can fail to govern the outcome anyone cares about. Both happened here, with a third failure — the implementation not executing the counted algorithm — between them. The remedy is cheap: report executed operation counts beside measured latency, name the baseline, and state the envelope. We release the harness, the corrected implementation, the pre-registration and the seven benchmarking pitfalls (Appendix A.13).

Broader Impact Statement

This work is a measurement and correction study of a computational-efficiency claim. It introduces no new capability and no new data. Its intended effect is to reduce wasted compute in the design of multi-agent communication architectures by discouraging optimisation against an operation count that does not predict runtime.

References

Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. *International Conference on Machine Learning (ICML)*, pp. 22–31, 2017. URL https://proceedings.mlr.press/v70/achiam17a.html.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document Transformer. *arXiv preprint*, 2020. URL https://arxiv.org/abs/2004.05150.

Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with Performers. *International Conference on Learning Representations (ICLR)*, 2021. URL https://arxiv.org/abs/2009.14794.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-Awareness. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html.

Abhishek Das, Théophile Gervet, Joshua Romoff, Dhruv Batra, Devi Parikh, Michael Rabbat, and Joelle Pineau. TarMAC: Targeted multi-agent communication. *International Conference on Machine Learning (ICML)*, 2019. URL https://arxiv.org/abs/1810.11187.

William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. URL https://jmlr.org/papers/v23/21-0998.html.

Ankit Gupta, Guy Dar, Shaya Goodman, David Ciprut, and Jonathan Berant. Memory-efficient Transformers via top-k attention. In *Proceedings of the Second Workshop on Simple and Efficient Natural Language Processing (SustaiNLP)*, pp. 39–52. Association for Computational Linguistics, November 2021. doi: 10.18653/v1/2021.sustainlp-1.5. URL https://aclanthology.org/2021.sustainlp-1.5/.

Mohammed Hamdan, Fares Ghaleb, Ahmed Bali, and Mohamed Cheriet. MARL-IoTP: Heterogeneous multi-agent learning for perception-aware edge orchestration. In *Fifth International Conference on Pattern Recognition and Artificial Intelligence (PRAI)*, 2026. doi: 10.2139/ssrn.6517702. URL https://ssrn.com/abstract=6517702. Accepted.

Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with Gumbel-Softmax. *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1611.01144.

Jiechuan Jiang and Zongqing Lu. Learning attentional communication for multi-agent cooperation. *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. URL https://arxiv.org/abs/1805.07733.

Jiechuan Jiang, Chen Dun, Tiejun Huang, and Zongqing Lu. Graph convolutional reinforcement learning. *International Conference on Learning Representations (ICLR)*, 2020. URL https://arxiv.org/abs/1810.09202.

Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient Transformer. *International Conference on Learning Representations (ICLR)*, 2020. URL https://arxiv.org/abs/2001.04451.

Xunhao Lai, Weiqi Xu, Yufeng Yang, Qiaorui Chen, Yang Xu, Lunbin Zeng, Xiaolong Li, Haohai Sun, Haichao Zhu, Vito Zhang, Jinkai Hu, Jiayao Li, Rui Gao, Zekun Li, Songquan Zhu, Jingkai Zhou, and Pengyu Zhao. MiniMax sparse attention. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2606.13392. URL https://arxiv.org/abs/2606.13392.

André F. T. Martins and Ramón Fernandez Astudillo. From softmax to sparsemax: A sparse model of attention and multi-label classification. *International Conference on Machine Learning (ICML)*, 2016. URL https://arxiv.org/abs/1602.02068.

Georgios Papoudakis, Filippos Christianos, Lukas Schäfer, and Stefano V. Albrecht. Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks. *Advances in Neural Information Processing Systems Datasets and Benchmarks Track (NeurIPS)*, 2021. URL https://arxiv.org/abs/2006.07869.

Ben Peters, Vlad Niculae, and André F. T. Martins. Sparse sequence-to-sequence models. *Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1504–1519, 2019. doi: 10.18653/v1/P19-1146. URL https://aclanthology.org/P19-1146/.

Markus N. Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint*, 2021. URL https://arxiv.org/abs/2112.05682.

Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with Routing Transformers. *Transactions of the Association for Computational Linguistics (TACL)*, 9: 53–68, 2021. doi: 10.1162/tacl_a_00353. URL https://aclanthology.org/2021.tacl-1.4/.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1701.06538.

Amanpreet Singh, Tushar Jain, and Sainbayar Sukhbaatar. Learning when to communicate at scale in multiagent cooperative and competitive tasks. *International Conference on Learning Representations (ICLR)*, 2019. URL https://arxiv.org/abs/1812.09755.

Adam Stooke, Joshua Achiam, and Pieter Abbeel. Responsive safety in reinforcement learning by PID Lagrangian methods. *International Conference on Machine Learning (ICML)*, pp. 9133–9143, 2020. URL https://proceedings.mlr.press/v119/stooke20a.html.

Sainbayar Sukhbaatar, Arthur Szlam, and Rob Fergus. Learning multiagent communication with backpropagation. *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. URL https://arxiv.org/abs/1605.07736.

Chuangchuang Sun, Macheng Shen, and Jonathan P. How. Scaling up multiagent reinforcement learning for robotic systems: Learn an adaptive sparse communication graph. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 11755–11762, 2020. doi: 10.1109/IROS45743.2020.9341303. URL https://arxiv.org/abs/2003.01040.

J. K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, Niall L. Williams, Yashas Lokesh, and Praveen Ravi. PettingZoo: Gym for multi-agent reinforcement learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. URL https://arxiv.org/abs/2009.14471.

Muning Wen, Jakub Grudzien Kuba, Runji Lin, Weinan Zhang, Ying Wen, Jun Wang, and Yaodong Yang. Multi-agent reinforcement learning is a sequence modeling problem. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2205.14953.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785. URL https://doi.org/10.1145/1498765.1498785.

Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative, multi-agent games. *Advances in Neural Information Processing Systems Datasets and Benchmarks Track (NeurIPS)*, 2022. URL https://arxiv.org/abs/2103.01955.

Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 23078–23097. Association for Computational Linguistics, 2025. URL https://aclanthology.org/2025.acl-long.1126/.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big Bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL https://arxiv.org/abs/2007.14062.

A Appendix

A.1 Machine balance and arithmetic intensity, tabulated

Table 4 gives the measured machine balance and the analytic intensities plotted in Figure 2.

Table 4: Measured machine balance, and analytic arithmetic intensity of the two paths at batch 256, d=16, $k = 25\%$ of peers. The gathered path (AI $\approx$ 0.64–0.66) is below every ridge estimate for either device at every N, and so is memory-bound throughout. The dense path is memory-bound on the GPU at every N; on the CPU it exceeds the 1.35 FLOP/byte estimate at every N we measure, but exceeds the higher 3.68 estimate only at $N \geq 192$. The CPU ridge is quoted at the lower end of the range described in the text, since that is the value the probe returns on the run archived with these numbers.

Device	**Achievable GEMM**	**Achievable bandwidth**	**Ridge point**
RTX PRO 6000 Blackwell (`fp32`)	75.3 TFLOP/s	1444 GB/s	52.1 FLOP/byte
x86_64 CPU (`fp32`, cache-resident)	1.75 TFLOP/s	1300 GB/s	1.35 FLOP/byte
N	**dense AI**	**gathered AI**	**ratio**
12	1.85	0.64	0.34×
48	3.23	0.66	0.20×
192	3.98	0.66	0.17×
512	4.18	0.66	0.16×

A.2 Dispersion behind every headline number

Table 1 reports min over interleaved samples, on the grounds that contention can only add time. That statistic is trustworthy only if the underlying sample was not wildly dispersed, so we report the median, the interquartile range and the spread of per-run medians for the same cells, and then state in Table 5 how much the headline ratios depend on the choice.

Across the full `fp32`, batch-256 sweep the median-to-minimum ratio runs from 1.00× to 1.06×, the largest being unfused dense at N=192, whose inter-run spread of 5.1% is the widest in the campaign. No headline number therefore rests on an outlying fast sample. Recomputing every headline ratio from medians rather than minima gives 1.15, 1.15, 1.16, 1.14, 1.59, 2.79, 2.45, 3.28 for $N = 6 \ldots 512$, against 1.15, 1.16, 1.17, 1.15, 1.61, 2.90, 2.46, 3.28 from minima: every cell moves by less than 3.6% and all eight remain above 1. No conclusion in the paper depends on the choice of statistic. The two statistics do have different minima, and the main text's "smallest ratio observed is 1.15×" refers throughout to the minimum-over-samples definition; the median-based series dips to 1.14 at N=48, stated here so the two definitions are not read as one number. The complete table for all N and all arms, with the raw per-run medians, is in the released artifacts.

Table 5: Forward-pass dispersion, `fp32`, batch 256, d=16, idle device, for the two smallest N in the sweep. Spread is the range of per-run medians across seven repeated timed blocks.

N	arm	min (ms)	median (ms)	IQR (ms)	median/min	spread
6	fused dense	0.1199	0.1239	0.0021	1.03	0.9%
	unfused dense	0.1266	0.1307	0.0031	1.03	0.9%
	MASKED	0.1751	0.1796	0.0044	1.03	0.7%
	GATHERED	0.1460	0.1498	0.0029	1.03	0.4%
	gate	0.3937	0.4036	0.0095	1.03	0.8%
12	fused dense	0.1213	0.1241	0.0019	1.02	0.2%
	unfused dense	0.1271	0.1316	0.0029	1.04	0.4%
	MASKED	0.1751	0.1809	0.0041	1.03	0.3%
	GATHERED	0.1476	0.1514	0.0027	1.03	0.1%
	gate	0.4066	0.4157	0.0097	1.02	0.7%

Table 6: Best-k gathered latency divided by *unfused* dense, GPU, `fp32`, eager. Values < 1 would indicate the gathered path winning; none do. Against the fused baseline one cell falls below 1 (d=128, B=4096, N=12, ratio 0.86). "skip" marks cells whose activation memory exceeded the budget (at d=256, B=4096, N=512 the gathered value tensor alone is ≈273 GiB) and which are recorded as skipped rather than dropped.

d	batch	N=12	N=48	N=192	N=512
32	32	1.17	1.18	1.18	2.19
32	256	1.18	1.19	1.83	2.40
32	4096	1.35	1.97	1.48	2.33
64	32	1.17	1.19	1.22	2.29
64	256	1.19	1.23	1.75	2.44
64	4096	1.49	1.84	1.47	2.34
128	32	1.17	1.19	1.23	1.81
128	256	1.19	1.24	1.51	2.10
128	4096	1.45	1.91	1.49	skip
256	32	1.18	1.18	1.18	1.44
256	256	1.17	1.25	1.37	1.70
256	4096	1.26	1.57	1.45	skip

A.3 Crossover surface, tabulated

Table 6 gives the numbers behind Figure 3.

Reading down the d axis at N=512 shows the trend discussed in Section 4.3: $2.19 \to 2.29 \to 1.81 \to 1.44$ at B=32 and $2.40 \to 2.44 \to 2.10 \to 1.70$ at B=256. The two cells that would continue the trend at B=4096 are the ones that exceeded memory.

A.4 Per-stage decomposition, tabulated

Table 7 gives the per-stage numbers behind Figure 4.

A.5 Verification of the corrected implementation

The claim that GATHERED and MASKED compute the same function while executing different work is load-bearing, so it is pinned by regression tests rather than asserted:

- **Functional equivalence.** Maximum absolute deviation between the two paths is $< 10^{-10}$ in `float64` for $N \in \{6, 12, 24\}$ and $k \in \{1, 2, 3, N-1\}$.

Table 7: Per-stage latency (ms, min over interleaved samples), batch 256, d=16. The aggregation "saving" is *negative* at every N: the sparse contraction is slower than the dense matmul it replaces, despite performing $\approx 4\times$ fewer multiply–accumulates. Overhead is $\texttt{topk} + \texttt{gather} + \texttt{softmax}_k - \texttt{softmax}_N$ and so is not a sum of the columns shown.

N	k	`topk`	`gather`	`agg_dense`	`agg_sparse`	saving	overhead
12	2	0.014	0.015	0.017	0.019	−0.003	+0.029
48	11	0.028	0.015	0.017	0.022	−0.005	+0.042
96	23	0.069	0.028	0.025	0.056	−0.031	+0.095
192	47	0.258	0.148	0.059	0.146	−0.087	+0.383
384	95	1.078	0.561	0.178	0.496	−0.318	+1.480
512	127	3.426	0.999	0.249	0.844	−0.595	+4.136

- **Distinct executed work.** With N=48, k=4, B=8, d=16 and input width 32, a FLOP counter reports 2,555,904 for dense and MASKED — identical, because the counter scores matrix products and MASKED issues the same two as dense — against 2,015,232 for GATHERED. The difference, 540,672, is exactly the aggregation term $2BNd(N-k)$: two FLOPs per multiply–add, which is the counter's convention rather than the MAC convention of Section 3.2, and the factor cancels in every ratio. MASKED's selection, masking and renormalisation are elementwise and are not matrix products, so they add executed work that this counter does not see; the equality above is therefore an equality of aggregation MACs, not of total executed work.
- **No dense aggregation in the sparse path.** Tracing every `torch.matmul` confirms GATHERED issues no $(B, N, N) \times (B, N, d)$ product and does issue $(B, N, 1, k) \times (B, N, k, d)$.
- **The score matmul survives in all modes.** Every top-k variant still issues the full $(B, N, d) \times (B, d, N)$ product, which is assumption A2 of Proposition 1 made mechanical.
- **The ceiling holds numerically.** For $N \in \{6, \ldots, 512\}$ the ratio $C_{\text{dense}}/C_{\text{top-}k}$ is verified < 2, and the structural variant of Proposition 2 is verified to track N/k in the zero-selector-cost case, $C_{\text{sel}} = 0$, which is the only case a pure operation count can pin: a nonzero C_{sel} moves the ratio below N/k by exactly the displayed formula.
- **Selection order is irrelevant.** `topk` with `sorted=False`, which the gathered path uses, agrees with the sorted call to 5.4×10^{-7} in `fp32` at every (N, k) tested, confirming that the softmax and gather are permuted consistently.
- **Which backend serves the fused arm.** A probe forces each SDPA backend in turn, at both dtypes and at $N \in \{12, 192, 512\}$, under no mask, an additive float mask and a Boolean mask, and the profiler records the kernel the default dispatcher actually launches. FlashAttention is rejected under any non-null mask, in both the additive-float and the Boolean form, and in `fp32` under any input; the fused arm runs `fmha_cutlassF`, PyTorch's memory-efficient CUTLASS kernel, at both dtypes. On the same unmasked `bf16` shape, forcing FlashAttention rather than the memory-efficient backend gives 0.0149 against 0.0223 ms at N=192 and 0.0388 against 0.0989 ms at N=512, so a baseline able to reach FlashAttention would widen the fused gap we report rather than narrow it. The log is released.

A.6 Activation memory of the evaluated gate

Figure 5 plots the peak activation memory discussed in Section 4.5.

A.7 The CPU allocator control

The CPU numbers of Section 4.6 required pinning the heap before they measured attention rather than the allocator. Dense, masked and gathered each allocate and free several multi-MiB (B, N, N) temporaries per

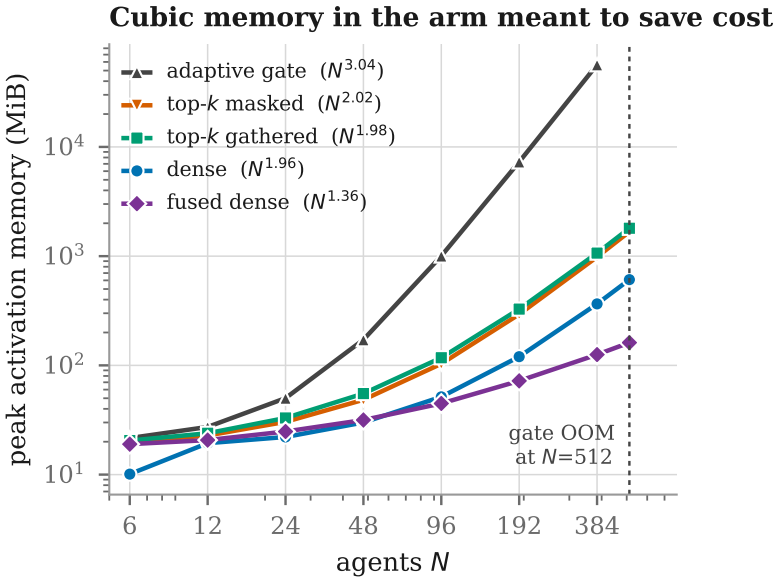


Figure 5: Peak activation memory against agent count, batch 256, with the offset-corrected fitted exponent beside each arm. The evaluated gate implementation grows cubically and is the first arm to exhaust the device; gathering saves no memory over masking, because the gathered value tensor replaces the zeros it avoids.

forward; the default glibc allocator returns those pages to the kernel, and they are re-faulted on the next call. We raise glibc's `MALLOC_TOP_PAD_` so that freed temporaries are retained instead, and count minor faults per forward from the process's own resource usage across the timed region.

Because the tuning is part of the measurement rather than a detail of it, the harness records the allocator environment, the thread counts, the affinity mask and the fault totals in each run's metadata, and the superseded un-pinned run is retained beside the pinned one. The effect it removes is large and unequal across arms: at N=192 the un-pinned run takes $\approx$ 37,000 extra minor faults per forward and inflates latency by up to 8.8×, and at N=48 it inverted the dense/gathered ordering — gathered appeared 0.81× unfused dense un-pinned, against 3.13× once the fault storm is removed. That was the only cell in this study, on either device, where a gathered path appeared to beat a dense one on equal terms.

A.8 Drift control, tabulated

Table 8 gives the reproducibility figures discussed in Section 4.7.

Table 8: Reproducibility of the headline sweep. Ratios — the quantity every claim is stated in — move by less than half as much as absolute latency at the median, though under load their worst case is comparable. Entries are median and maximum absolute change across the eight N cells.

	latency $\lvert\Delta\rvert$		**ratio** $\lvert\Delta\rvert$	
Condition	median	max	median	max
Same device, idle, later run	0.7%	2.6%	**0.4%**	**1.4%**
Same device, concurrent GPU load	67.9%	114.2%	33.7%	115.9%

A.9 Pre-registration chronology

The analysis plan of Section 5 was committed when 8 of the 40 tier-2–4 runs had produced a final evaluation — all five seeds of `gathered-`k=2 and three of five of `random-`k=2 — and before any tier-2–4 return had

been inspected. Commit history establishes the ordering: tier-1 artifacts at 12:18, the plan at 12:56, tier-2–4 artifacts at 19:11 the same day. The tier-1 arms were complete and had been inspected, which is why the tier-1 gate is excluded from the inferential family.

The pre-registration document has two commits: the plan itself, and a later append to its deviations log. The diff between them touches only the deviations section; the endpoint, contrasts, test, effect sizes, interval and correction are byte-identical to the pre-unblinding version. The log records one item, the bootstrap/permutation tension discussed in Section 5. Timestamps establish ordering and nothing more — they cannot establish that a file on disk was never opened — so the plan's independence is stated as a claim about our conduct rather than as something the artifact proves, and data generation had demonstrably already begun.

A.10 The reporting-audit procedure

For each of the twelve papers in Table 9 we read the full text, including appendices and the official supplement where one exists, searched it for *latency*, *runtime*, *time*, *throughput*, *speed*, *wall-clock* and *FLOP/s*, and recorded (i) whether it makes a computational-efficiency or complexity claim about its mechanism, (ii) whether it reports any measured temporal quantity attributable to that mechanism, and (iii) the figure, table or sentence that settled the decision. A statement such as "training took a few days" does not qualify, since it is not attributable to the mechanism. The completed sheet is released with the artifacts, because a "no" is an absence claim and an absence claim is only as good as the search behind it. Scope matters for at least one entry: MAT's "yes" rests on a wall-clock figure in its appendix and would be a "no" on the main body alone.

A.11 Secondary training analysis

Two results we report but do not interpret. C4 has the one sign opposite to its pre-registered direction, and should be set aside for a reason independent of its p-value: neither N=12 arm learned the task. A uniform-random policy scores -758.9 under the same evaluation protocol, against -864.9 for the dense arm — worse than random, two of five seeds below -975 — and -717.9 for the gathered arm, so the $+147$ point estimate compares a diverged run with a barely learning one. Second, for C4 and C6 the bootstrap interval excludes zero while the permutation test does not. The pre-registration named the permutation test, so it governs; the disagreement is unsurprising at n=5, where only $5^5 = 3125$ distinct resamples exist per arm and the interval is coarse, discrete and of uncertain coverage. We treat it as descriptive and keep both on the page, so the reader sees the size of the estimated effect as well as the strength of the evidence for it.

Descriptively, and supporting no claim: the Lagrangian arm meets its budget but not by the intended route, its gate collapsing to expected $k \approx 1.01$ within the first 8% of training while the dual multiplier decays to zero; entmax settles at a support of 1.64 of 5 peers; and attention entropy falls from 0.35 nats over 5 peers to 0.12 over 2. The three sit on different supports and are reported only as an instrumentation check.

A.12 The partially observable environment

The variant used in Section 5 modifies `simple_spread` in one respect: for each agent i and peer j, the relative-position entry $\text{peer_rel}[i, j]$ is zeroed when $\|\text{peer_rel}[i, j]\|_2 > R$. Reward, action space, landmark observations, episode length and observation dimension are unchanged, and R=1.0 throughout.

Two properties of the variant should be stated because they bound what it can support. First, the mask is a zeroing rather than an explicit visibility flag, so a peer at relative position exactly $(0, 0)$ is not distinguishable from an absent one; measured under a uniform random policy, 0.06% of visible peers at N=6 and 0.15% at N=12 lie within 0.05 of the observer, which bounds how often the ambiguity can arise. Second, the visible *fraction* falls with N while the absolute number visible rises, because the world size is fixed; the two series are given in Section 5, and the consequence is a difficulty confound that rules out across-N return comparisons in this environment.

The communicating arms carry between 7.2% (N=12) and 19.0% (`simple_reference`) more trainable parameters than the communication-free MAPPO baseline, since the attention module adds parameters that the baseline has no analogue for. The positive controls therefore establish that the communicating architecture outperforms the baseline, not that the channel alone accounts for the difference. Within the pre-registered family the arms share an architecture and the comparison is tighter; C1 in particular holds k, executed FLOPs and every hyperparameter fixed.

A.13 Reproduction and provenance

The measurement campaign is training-free and completes in minutes. The released harness regenerates the tables in this paper from the raw JSON artifacts, and the training arms are driven by generated configuration files with the tier ordering enforced by the runner.

Two provenance caveats. The aggregate provenance record covers the earlier microbenchmark phases of this project, which ran on different hardware that is no longer available; run-level provenance for the training arms is in the per-run configuration files rather than in that record. The CPU numbers in Section 4.6 were taken with the glibc heap pinned; an earlier campaign that did not do so is superseded and retained in the artifacts with a note. The activation-memory measurement is reported from a run with the whole card free, since a shared-host repeat exhausts memory one N earlier for the gate; the fitted exponents for the other four arms are identical between the two runs.

Table 9: Type of evidence offered for efficiency claims, in the twelve audited papers (n=6 per group). "Measured temporal metric" means any measured time, speed, throughput or achieved FLOP/s attributable to the mechanism. This is an audit of reporting practice in a small, deliberately selected set — the papers we ourselves relied on — not a sample of either literature, and a "no" means the argument rests on operation counts or asymptotics, not that it is wrong.

Paper	**Efficiency claim**	**Type**	**Measured temporal metric?**
Multi-agent communication / attention			
CommNet (Sukhbaatar et al., 2016)	none explicit	—	n/a
TarMAC (Das et al., 2019)	compact messages	qualitative	no
IC3Net (Singh et al., 2019)	fewer messages	counts	no
ATOC (Jiang & Lu, 2018)	bounded group size	counts	no
DGN (Jiang et al., 2020)	bandwidth + complexity	asymptotic	no
MAT (Wen et al., 2022)	linear in N	asymptotic	**yes**
Efficient attention / sparse mixture-of-experts			
Reformer (Kitaev et al., 2020)	$\mathcal{O}(L \log L)$	asymptotic	**yes**
Longformer (Beltagy et al., 2020)	$\mathcal{O}(L)$	asymptotic	**yes**
BigBird (Zaheer et al., 2020)	$\mathcal{O}(L)$	asymptotic	no
Performer (Choromanski et al., 2021)	$\mathcal{O}(L)$	asymptotic	**yes**
Switch (Fedus et al., 2022)	FLOPs-matched	FLOPs	**yes**
Sparse MoE (Shazeer et al., 2017)	large capacity, low cost	FLOPs	**yes**